

Preference-Aware Coefficient Correction for Rewarded-Soups-Style Model Merging

Nils

June 24, 2026

Contents

1	Introduction	3
2	Background and Related Work	4
2.1	Multi-objective methods that inspire the coefficient mappings	4
3	Problem Formulation	6
3.1	Objectives and rewards	6
3.2	Base model, specialists, and task vectors	8
3.3	Relationship matrix from task-vector geometry	8
3.4	Preference-to-coefficient mapping	10
3.5	Baselines and evaluation	10
4	Method	14
4.1	General setup	14
4.2	MGDA-inspired coefficient mapping	14
4.3	PCGrad-inspired coefficient mapping	15
4.4	CAGrad-inspired coefficient mapping	16
5	Practical LoRA-Based Implementation	18
5.1	LoRA definition	18
5.2	LoRA geometry and merging	18
6	Experimental Setup	20
6.1	Candidate datasets	20
6.2	First prototype	20
7	Results	21
8	Discussion	22
9	Conclusion	23

Abstract

This document is a rough first draft. It does not report experiments.

Chapter 1

Introduction

This thesis studies preference-aware coefficient correction for model merging under a fixed interpolation family. The central question is whether a geometry-aware mapping

$$\lambda = f(p, R)$$

can improve coefficient selection relative to the direct baseline $\lambda = p$, while staying in the same Rewarded-Soups-style family.

This thesis is organized around three research questions; **RQ2** is the pivotal one, since it governs whether the others can be answered affirmatively.

RQ1 (Does geometry-aware correction help?)

Within the fixed Rewarded-Soups interpolation family $\mathcal{M} = \{\theta(\lambda) : \lambda \in \Delta_m\}$, can a conflict-aware mapping $\lambda = f(p, R)$ achieve higher preference-weighted utility U_p than the direct baseline $\lambda = p$, and than search-based selection at a matched compute budget?

RQ2 (Is R a valid proxy, and when does it help?)

To what extent does the parameter-geometric relationship matrix R predict reward-space behaviour? Because R never observes the rewards directly, quantifying the strength of this proxy — and the regimes in which it holds — is what determines when $f(p, R)$ can be expected to help, and whether reward-informed correction is needed to close any remaining gap.

RQ3 (How does the benefit scale with m ?)

Dense search over the simplex Δ_m becomes expensive as the number of objectives m grows. How does the advantage of a one-shot mapping $f(p, R)$ over budgeted search scale with m ?

Chapter 2

Background and Related Work

Rewarded Soups motivates the fixed linear interpolation family considered here [Ramé et al., 2023]. Task Arithmetic motivates the use of task vectors defined relative to a shared base model [Ilharco et al., 2022]. TIES and AdaMerging are nearby model-merging baselines [Yadav et al., 2023, Yang et al., 2023]. MGDA, PCGrad, and CAGrad are gradient-based multi-objective methods that here serve only as inspiration for one-shot, R -based coefficient mappings rather than their original training-time algorithms [Sener and Koltun, 2018, Yu et al., 2020, Liu et al., 2021]. LoRA is the practical adapter mechanism for the prototype implementation [Hu et al., 2021].

2.1 Multi-objective methods that inspire the coefficient mappings

For context, we briefly summarize the three multi-objective optimization methods that inspire the coefficient mappings defined in Chapter 4. In all three cases only the underlying geometric principle is reused; it is applied to the static relationship matrix R rather than to live per-step gradients, and no coefficient-space gradient descent is performed.

MGDA (Multiple Gradient Descent Algorithm) casts multi-task learning as multi-objective optimization [Sener and Koltun, 2018]. Given a set of per-task gradients, it selects the convex combination of minimum aggregated norm; the resulting update direction lies in the common descent cone, so no individual task loss increases to first order. Strongly conflicting tasks pull the combination weights toward a balancing point.

PCGrad (Gradient Surgery) resolves conflicts locally and pairwise [Yu et al., 2020]. Whenever two task gradients have a negative inner product (an interference), one gradient is projected onto the orthogonal complement of the other, removing only the conflicting component while leaving complementary directions intact. This contrasts with MGDA, which computes a single global balance.

CAGrad (Conflict-Averse Gradient Descent) optimizes the average objective while constraining the update to a trust region around the average gradient [Liu et al., 2021]. The constraint bounds the worst-case per-task decrease, trading off

average progress against worst-case protection.

In this thesis these ideas are transferred to one-shot coefficient selection: the gradient Gram matrix is replaced by the static relationship matrix R , conflict projections and trust regions are expressed in the R -induced geometry, and the output is a single coefficient vector $\lambda \in \Delta_m$ rather than an iterative descent trajectory.

TODO: Expand this chapter into a proper related-work review.

Chapter 3

Problem Formulation

In this chapter, $m \geq 2$ denotes the number of objectives and d the flattened parameter dimension. Table 3.1 summarizes the notation used throughout.

3.1 Objectives and rewards

Definition 3.1 (Objectives and rewards). Let $\Theta := \mathbb{R}^d$ denote the parameter space. For each objective $i \in \{1, \dots, m\}$, let

$$r_i : \Theta \rightarrow \mathbb{R}$$

be a reward function. Define

$$r(\theta) := (r_1(\theta), \dots, r_m(\theta)) \in \mathbb{R}^m.$$

If a loss notation is useful, define optionally

$$F_i(\theta) := -r_i(\theta).$$

This thesis does *not* define coefficient-space losses such as $F_i(\lambda)$.

Definition 3.2 (Preference vector). The user preference vector is

$$p \in \Delta_m, \quad \Delta_m := \{x \in \mathbb{R}^m : x_i \geq 0 \ \forall i, \ \mathbf{1}^\top x = 1\}.$$

Definition 3.3 (Preference-weighted utility). For fixed $p \in \Delta_m$, define

$$U_p(\theta) := \sum_{i=1}^m p_i r_i(\theta).$$

If reward scales differ, let $\tilde{r}_i(\theta)$ denote fixed normalized rewards and evaluate with

$$U_p(\theta) := \sum_{i=1}^m p_i \tilde{r}_i(\theta).$$

Remark 3.4 (Normalization). If normalized rewards are used, the normalization rule should be fixed before comparing coefficient-selection methods.

Table 3.1: Notation used throughout the thesis.

Symbol	Meaning	Type / dim.
<i>Models and task-vector geometry</i>		
θ_0	base model	$\mathbb{R}^d$
θ_i	specialist fine-tuned for objective i	$\mathbb{R}^d$
$\delta_i = \theta_i - \theta_0$	task vector / LoRA-adapter update	$\mathbb{R}^d$
$D = [\delta_1, \dots, \delta_m]$	stacked task vectors	$\mathbb{R}^{d \times m}$
$\theta(\lambda) = \theta_0 + D\lambda$	merged model	$\mathbb{R}^d$
$\mathcal{M} = \{\theta(\lambda) : \lambda \in \Delta_m\}$	interpolation family	—
<i>Preferences and coefficients</i>		
m	number of objectives	$\mathbb{N}$
d	parameter dimension ($\sim 10^6$)	$\mathbb{N}$
p	user preference vector	Δ_m
$\lambda = f(p, R)$	corrected merge coefficients	Δ_m
$\Delta_m = \{x \geq 0 : \mathbf{1}^\top x = 1\}$	probability simplex	—
<i>Relationship matrix</i>		
$R = D^\top D$	Gram relationship matrix	$\mathbb{R}^{m \times m}$, PSD
$\hat{R} = \hat{D}^\top \hat{D}$	cosine relationship matrix	$\mathbb{R}^{m \times m}$, PSD
<i>Rewards and evaluation metrics</i>		
$r_i, \tilde{r}_i$	reward i , normalized reward	scalar
$U_p(\theta(\lambda)) = \sum_i p_i \tilde{r}_i(\theta(\lambda))$	preference-weighted utility	scalar
ΔU_p	gain over the direct baseline	scalar
T_p^{norm}	weighted Tchebycheff (worst-case) metric	scalar
$\ \lambda - p\ $	preference faithfulness	scalar
$\mathcal{B} \subset \Delta_m$	finite search / candidate set	—
$\lambda_{\text{best}}^{\mathcal{B}}$	best coefficient found in $\mathcal{B}$	Δ_m
$\text{Gap}_p^{\mathcal{B}}$	within-family regret vs. $\mathcal{B}$	scalar
<i>Hyperparameters</i>		
ρ	faithfulness penalty (M1, M2, C2, C3)	> 0
c	trust-region radius (C1)	≥ 0
β	conflict-shrinkage strength (P1)	≥ 0
τ	soft-min temperature (C2)	> 0
ε	trust-region floor / numerical guard	> 0

3.2 Base model, specialists, and task vectors

Assumption 3.5 (Shared-base requirement). There exists a shared base model $\theta_0 \in \Theta$. For each objective $i \in \{1, \dots, m\}$, the specialist model $\theta_i \in \Theta$ is obtained from the same base model θ_0 , with the same parameterization and parameter ordering.

Definition 3.6 (Task vectors). For each specialist i , define

$$\delta_i := \theta_i - \theta_0 \in \mathbb{R}^d.$$

The shared-base requirement is necessary for the task-vector interpretation and the linear interpolation family used in this thesis.

Definition 3.7 (Rewarded-Soups-style interpolation family). For $\lambda \in \Delta_m$, define

$$\theta(\lambda) := \theta_0 + \sum_{i=1}^m \lambda_i \delta_i.$$

Equivalently, because $\sum_i \lambda_i = 1$ and $\delta_i = \theta_i - \theta_0$,

$$\theta(\lambda) = \sum_{i=1}^m \lambda_i \theta_i.$$

Define the fixed interpolation family

$$\mathcal{M} := \{\theta(\lambda) : \lambda \in \Delta_m\}.$$

This thesis operates only inside $\mathcal{M}$.

Remark 3.8 (Non-claim). Any empirical gain discussed later would concern only *selection of λ inside $\mathcal{M}$* . It would not imply a global Pareto-front improvement over all $\theta \in \Theta$.

3.3 Relationship matrix from task-vector geometry

Definition 3.9 (Task vectors and task-vector matrix). For each specialist i , define the task vector and collect the task vectors as columns:

$$\delta_i := \theta_i - \theta_0, \quad \Delta := [\delta_1, \dots, \delta_m] \in \mathbb{R}^{d \times m}.$$

Thus

$$\theta(\lambda) = \theta_0 + \Delta \lambda.$$

Definition 3.10 (Relationship matrix). The unnormalized Gram relationship matrix is

$$\mathbf{R} := \Delta^\top \Delta,$$

with scalar entries

$$R_{ij} = \delta_i^\top \delta_j.$$

If $\delta_i \neq 0$ for all i , define normalized task vectors and their matrix by

$$\hat{\delta}_i := \frac{\delta_i}{\|\delta_i\|_2}, \quad \hat{\Delta} := [\hat{\delta}_1, \dots, \hat{\delta}_m].$$

The cosine relationship matrix is then

$$\hat{\mathbf{R}} := \hat{\Delta}^\top \hat{\Delta}, \quad \hat{R}_{ij} = \cos(\delta_i, \delta_j) = \frac{\delta_i^\top \delta_j}{\|\delta_i\|_2 \|\delta_j\|_2}.$$

Remark 3.11 (PSD status). If $\mathbf{R} = \Delta^\top \Delta$, then $\mathbf{R}$ is positive semidefinite by construction. The normalized Gram matrix $\hat{\mathbf{R}} = \hat{\Delta}^\top \hat{\Delta}$ is positive semidefinite as well. An arbitrary non-Gram similarity matrix need not be positive semidefinite.

Definition 3.10 records only inner products among the task vectors; the reward gradients ∇r_i do not appear in it. To state precisely what $\mathbf{R}$ is a proxy for, we first identify the object that governs reward behavior to first order.

Definition 3.12 (Reward-sensitivity matrix). For differentiable rewards r_i , define the reward-sensitivity matrix $\mathbf{H} \in \mathbb{R}^{m \times m}$ by

$$H_{ij} := \langle \nabla r_i(\theta_0), \delta_j \rangle.$$

For $\theta(\lambda) = \theta_0 + \Delta \lambda$, a first-order expansion of the preference-weighted utility around θ_0 gives

$$U_{\mathbf{p}}(\theta(\lambda)) - U_{\mathbf{p}}(\theta_0) = \mathbf{p}^\top \mathbf{H} \lambda + o(\|\Delta \lambda\|_2) \approx \mathbf{p}^\top \mathbf{H} \lambda.$$

Indeed, $\nabla U_{\mathbf{p}}(\theta_0) = \sum_i p_i \nabla r_i(\theta_0)$. Hence reward behavior is governed to first order by $\mathbf{H}$, not directly by $\mathbf{R}$: the scalar H_{ij} is the first-order change in reward i induced by moving along specialist direction δ_j .

Assumption 3.13 (Linear short-horizon proxy). Suppose that each task vector is approximately a short gradient step on its own reward, with a common step scale $\eta > 0$:

$$\delta_j \approx \eta \nabla r_j(\theta_0).$$

Then

$$H_{ij} \approx \eta \langle \nabla r_i(\theta_0), \nabla r_j(\theta_0) \rangle, \quad R_{ij} \approx \eta^2 \langle \nabla r_i(\theta_0), \nabla r_j(\theta_0) \rangle,$$

and therefore, for the unnormalized Gram matrix,

$$H_{ij} \approx \frac{1}{\eta} R_{ij}, \quad \mathbf{H} \approx \frac{1}{\eta} \mathbf{R}.$$

Under this assumption, the symmetric, gradient-free matrix $\mathbf{R}$ can serve as a proxy for $\mathbf{H}$.

Remark 3.14 (Validity, locality, and the limits of $\mathbf{R}$). The proxy in Assumption 3.13 is local and first order. Its remainder is of order $o(\|\Delta \lambda\|_2)$, so the approximation is reliable only near the base model; far from θ_0 , the linearization and the predictive value of $\mathbf{R}$ may break down. Moreover, $\mathbf{H}$ is generally asymmetric: the effect of δ_j on reward i need not equal the effect of δ_i on reward j . In contrast, $\mathbf{R} = \Delta^\top \Delta$ is symmetric by construction and cannot represent such directed help or harm effects. Fully trained adapters or RLHF specialists may also violate $\delta_j \approx \eta \nabla r_j(\theta_0)$, because their net displacement need not be proportional to the initial reward gradient. The validity of $\mathbf{R}$ as a proxy for $\mathbf{H}$ must therefore be tested empirically rather than assumed.

3.4 Preference-to-coefficient mapping

Definition 3.15 (Preference-to-coefficient mapping). The main object of study is

$$f : \Delta_m \times \mathbb{R}^{m \times m} \rightarrow \Delta_m, \quad \lambda = f(p, R).$$

The direct preference baseline is

$$\lambda_{\text{direct}} := p.$$

Even though p and λ lie in the same simplex, they are conceptually different:

- p is a preference vector in reward/objective space,
- λ is a merge coefficient vector in parameter/model space.

Definition 3.16 (Coefficient-selection problem). Given $p \in \Delta_m$ and a fixed relationship matrix R , the thesis studies whether a geometry-aware correction

$$\lambda = f(p, R)$$

can improve coefficient selection relative to $\lambda = p$, while keeping $\theta(\lambda)$ inside the same interpolation family $\mathcal{M}$. The intended pipeline is

$$(\delta_i)_{i=1}^m \longrightarrow R \longrightarrow \lambda = f(p, R) \longrightarrow \theta(\lambda).$$

Remark 3.17 (Selection cost is independent of the parameter dimension). Every coefficient mapping $f(p, R)$ operates only on the relationship matrix $R \in \mathbb{R}^{m \times m}$ and the coefficient vector $\lambda \in \mathbb{R}^m$. The high-dimensional parameter space enters exactly once, when $R = D^\top D$ is formed from the task vectors at cost $O(m^2 d)$; this is computed a single time and reused for every preference. Thereafter, solving for $\lambda = f(p, R)$ — and re-solving for any new preference p — costs $\text{poly}(m)$ and is *independent of* d . The parameter dimension d (millions, for LoRA adapters) is touched again only to realize the final merge $\theta(\lambda) = \theta_0 + D\lambda$ for a chosen λ . By contrast, search over $\mathcal{B}$ requires $|\mathcal{B}|$ separate merges and reward evaluations, each touching d . This is the precise sense in which the proposed mappings are one-shot and cheap: the selection problem is solved entirely in the m -dimensional space.

3.5 Baselines and evaluation

Definition 3.18 (Baselines). The following baselines are considered:

$$\lambda_i^{\text{unif}} = \frac{1}{m}, \quad \lambda^{\text{direct}} = p.$$

In addition, one may use random search, Dirichlet sampling, or Sobol-style search over Δ_m .

Remark 3.19 (Finite-search notation). To avoid a clash with LoRA matrices B_i , the finite evaluated coefficient set is denoted by $\mathcal{B} \subset \Delta_m$.

Definition 3.20 (High-budget best-found reference). Given finite $\mathcal{B} \subset \Delta_m$, define

$$\lambda_{\text{best}}(p) := \arg \max_{\lambda \in \mathcal{B}} U_p(\theta(\lambda)).$$

This is a finite-search reference, not a true oracle over all of Δ_m .

Definition 3.21 (Evaluation metrics). All evaluation metrics are defined for coefficient vectors $\lambda \in \Delta_m$ and therefore evaluate only models inside the fixed family

$$\mathcal{M} = \{\theta(\lambda) : \lambda \in \Delta_m\}.$$

They are not global Pareto-front metrics.

Let $\tilde{r}_i(\theta(\lambda))$ denote the evaluation reward for objective i . If objective scales are already comparable, we set $\tilde{r}_i = r_i$. Otherwise, the normalization rule is fixed before comparing methods. A typical min-max normalization uses constants $a_i < b_i$, obtained from known reward ranges or from a separate calibration set, and defines

$$\tilde{r}_i(\theta) := \frac{r_i(\theta) - a_i}{b_i - a_i}.$$

If the normalization constants are instead estimated from the finite evaluated set $\mathcal{B}$, this choice must be reported explicitly because the same set is also used to define the best-found reference.

For a fixed preference vector $p \in \Delta_m$, the individual objective scores are

$$\tilde{r}_i(\theta(\lambda)), \quad i = 1, \dots, m.$$

The preference-weighted utility is

$$U_p(\theta(\lambda)) := \sum_{i=1}^m p_i \tilde{r}_i(\theta(\lambda)),$$

where larger values are better for the fixed preference p .

The direct preference baseline is $\lambda = p$. To avoid confusing the preference vector with a model parameter, we write the baseline model explicitly as $\theta(\lambda = p)$. The utility improvement over direct preference interpolation is

$$\Delta U_p(\lambda_{\text{method}}) := U_p(\theta(\lambda_{\text{method}})) - U_p(\theta(\lambda = p)).$$

Thus $\Delta U_p > 0$ means that the method improves over the direct baseline for the same preference p .

The best-found reference is computed separately for each preference:

$$\lambda_{\text{best}}^{\mathcal{B}}(p) := \arg \max_{\lambda \in \mathcal{B}} U_p(\theta(\lambda)).$$

The superscript $\mathcal{B}$ emphasizes that this is only the best point found in the finite evaluated set. The within-family utility gap is

$$\text{Gap}_p^{\mathcal{B}}(\lambda_{\text{method}}) := U_p(\theta(\lambda_{\text{best}}^{\mathcal{B}}(p))) - U_p(\theta(\lambda_{\text{method}})).$$

A smaller gap is better. If $\lambda_{\text{method}} \in \mathcal{B}$, then $\text{Gap}_p^{\mathcal{B}}(\lambda_{\text{method}}) \geq 0$.

Preference faithfulness is reported as a diagnostic trade-off, not as a reward metric. It can be measured by

$$\|\lambda_{\text{method}} - p\|_1 \quad \text{or} \quad \|\lambda_{\text{method}} - p\|_2.$$

An optional geometry-aware diagnostic is

$$D_{\bar{R}}(\lambda_{\text{method}}, p) := (\lambda_{\text{method}} - p)^\top \bar{R}(\lambda_{\text{method}} - p),$$

where $\bar{R} \succeq 0$ is the matrix used for distance evaluation. If the selected relationship matrix is already a Gram matrix, one may use either $\bar{R} = R = D^\top D$ or $\bar{R} = \hat{R} = \hat{D}^\top \hat{D}$. If the selected relationship matrix is not guaranteed to be symmetric positive semidefinite, then $\bar{R}$ should be obtained by symmetrizing and projecting to the positive semidefinite cone. If $\bar{R} = D^\top D$, then

$$D_{\bar{R}}(\lambda_{\text{method}}, p) = \|D(\lambda_{\text{method}} - p)\|_2^2.$$

For a Tchebychev-style worst-case shortfall metric, define observed best and worst normalized rewards over the finite evaluated set

$$z_i^{\text{best}} := \max_{\lambda \in \mathcal{B}} \tilde{r}_i(\theta(\lambda)), \quad z_i^{\text{worst}} := \min_{\lambda \in \mathcal{B}} \tilde{r}_i(\theta(\lambda)).$$

For numerical stability, let $\varepsilon_T > 0$. Define the active objective set

$$A_{\varepsilon_p}(p) := \{i \in \{1, \dots, m\} : p_i > \varepsilon_p\},$$

with a small threshold $\varepsilon_p \geq 0$. The normalized Tchebychev-style metric is

$$T_p^{\text{norm}}(\lambda) := \max_{i \in A_{\varepsilon_p}(p)} p_i \left[\frac{z_i^{\text{best}} - \tilde{r}_i(\theta(\lambda))}{z_i^{\text{best}} - z_i^{\text{worst}} + \varepsilon_T} \right].$$

Smaller values are better. This convention multiplies the normalized shortfall by p_i , so shortfalls in high-preference objectives are penalized more strongly. Objectives with zero preference do not affect this metric. Objectives with negligible observed range should be handled carefully because the denominator can amplify small absolute differences.

The Tchebychev improvement over the direct baseline is

$$\Delta T_p^{\text{norm}}(\lambda_{\text{method}}) := T_p^{\text{norm}}(\lambda = p) - T_p^{\text{norm}}(\lambda_{\text{method}}).$$

A positive value means that the method reduces the worst-case preference-weighted shortfall relative to $\lambda = p$.

Finally, evaluation is performed over a set of preference vectors

$$\mathcal{P} := \{p^{(1)}, \dots, p^{(K)}\} \subset \Delta_m.$$

Metrics such as ΔU_p , $\text{Gap}_p^{\mathcal{B}}$, T_p^{norm} , and preference-faithfulness distances are reported per preference and summarized over $\mathcal{P}$ using mean, median, and robust statistics.

Remark 3.22 (Tchebychev weighting convention). The metric T_p^{norm} follows the standard weighted Tchebycheff scalarization [Miettinen, 1999], in which the weight *multiplies* the deviation from the ideal reference point,

$$g_{\text{tch}}(\theta \mid w, z^*) = \max_{i=1,\dots,m} \{ w_i |f_i(\theta) - z_i^*| \}.$$

Choosing $w_i = p_i$ is motivated by three considerations. First, preference consistency: a shortfall on a high-preference objective should be penalized more than one on a low-preference objective. Second, consistency with the primary metric U_p , which also weights rewards by p_i ; both the average-case and the worst-case metric then share the same preference perspective. Third, the zero-preference case is handled cleanly, since objectives with $p_i \leq \varepsilon_p$ are excluded through $A_{\varepsilon_p}(p)$, whereas an inverse $1/p_i$ weighting would diverge there. The inverse convention $w_i \propto 1/p_i$ arises only when scalarization is used to *place* a solution along a target direction p on the Pareto front, as in MORL/MOEA/D decomposition; this is a solution-selection technique rather than an evaluation principle, and is not used here.

Remark 3.23 (Evaluation scope). All quantities above compare only coefficient choices inside the fixed family $\mathcal{M}$ or the finite evaluated set $\mathcal{B}$. The notation $\lambda_{\text{best}}^{\mathcal{B}}(p)$ should not be interpreted as a global optimum, and improvements in ΔU_p or ΔT_p^{norm} do not imply global Pareto-front improvement.

Remark 3.24 (A negative $\text{Gap}_p^{\mathcal{B}}$ is a success, not an anomaly). The reference $\lambda_{\text{best}}^{\mathcal{B}}(p)$ is the best coefficient within a *finite* search set $\mathcal{B}$ of budget $|\mathcal{B}|$, not a ground-truth optimum over Δ_m . Consequently $\text{Gap}_p^{\mathcal{B}}$ can be negative: this happens exactly when a method finds a coefficient that no point in $\mathcal{B}$ matches, i.e. when $f(p, R)$ outperforms budgeted search. A negative $\text{Gap}_p^{\mathcal{B}}$ should therefore be read as evidence for the efficiency claim of RQ3, and $\text{Gap}_p^{\mathcal{B}}$ should always be reported together with the budget $|\mathcal{B}|$ that defines the reference.

Chapter 4

Method

This chapter defines one-shot mappings $\lambda = f(p, R)$ that use only the preference vector p and a static relationship matrix R . No coefficient-space gradients are introduced. We define seven mappings:

M1, M2, P1, P2, C1, C2, C3.

They are grouped by the multi-objective method that inspires them: *M1* and *M2* (MGDA-inspired), *P1* and *P2* (PCGrad / conflict-inspired), and *C1*, *C2*, and *C3* (CAGrad-inspired).

4.1 General setup

The relationship-matrix construction and its reward-proxy interpretation are given in Section 3.3; the limitations of this proxy are summarized in Remark 3.14. For optimization-based methods below, assume that the relationship matrix supplied to the mapping is symmetric and positive semidefinite. The canonical choices are the unnormalized matrix $R = D^\top D$ and the cosine matrix $\hat{R} = \hat{D}^\top \hat{D}$. For compactness, the method definitions use R as the generic matrix argument; when cosine geometry is selected, this argument is instantiated as $R \leftarrow \hat{R}$. The current implementation uses $\hat{R}$.

4.2 MGDA-inspired coefficient mapping

Definition 4.1 (M1: MGDA-inspired one-shot mapping). Given $p \in \Delta_m$, $R \in \mathbb{R}^{m \times m}$, and $\rho > 0$, define

$$\lambda_{\text{M1}} := \arg \min_{\lambda \in \Delta_m} \lambda^\top R \lambda + \rho \|\lambda - p\|_2^2.$$

This is inspired by min-norm balancing, but adapted to coefficient selection rather than gradient descent.

Remark 4.2 (Possible limitation). Depending on how informative R is, this objective may encourage cancellation or move too strongly toward low-norm combinations.

Definition 4.3 (M2: Preference-weighted α -MGDA variant). Let

$$G_p := \text{Diag}(p) R \text{Diag}(p), \quad u := \frac{1}{m} \mathbf{1}.$$

Define

$$\alpha^* := \arg \min_{\alpha \in \Delta_m} \alpha^\top G_p \alpha + \rho \|\alpha - u\|_2^2,$$

and then

$$\lambda_{\text{M2}} := \frac{\alpha^* \odot p}{\mathbf{1}^\top (\alpha^* \odot p)}.$$

This variant corrects p multiplicatively instead of replacing it outright. It is well defined for any $p \neq 0$; objectives with $p_i = 0$ receive coefficient zero.

4.3 PCGrad-inspired coefficient mapping

We consider two conflict-based mappings: a closed-form shrinkage (P1) that requires no solver, and a projection-based reconstruction (P2) that more directly mirrors PCGrad’s gradient surgery.

Definition 4.4 (P1: Conflict-weighted closed-form shrinkage). Given $p \in \Delta_m$, $R \in \mathbb{R}^{m \times m}$, and $\beta > 0$, define the per-objective conflict load

$$\kappa_i := \sum_{j \neq i} \max(0, -R_{ij}),$$

the shrinkage factors

$$s_i := \frac{1}{1 + \beta \kappa_i},$$

and the raw weights $\tilde{\lambda}_i := p_i s_i$. The coefficient vector is the simplex normalization

$$\lambda_{\text{P1}} := \frac{\tilde{\lambda}}{\mathbf{1}^\top \tilde{\lambda}}.$$

This is a closed-form mapping: objectives that conflict strongly with others (large κ_i) are down-weighted relative to their preference p_i , while objectives that are aligned or complementary keep $s_i \approx 1$.

Remark 4.5 (Scale of β and R). The conflict load κ_i inherits the scale of the selected relationship matrix. With the cosine matrix $\hat{R} = \hat{D}^\top \hat{D}$, the off-diagonal entries lie in $[-1, 1]$ and $\kappa_i \in [0, m - 1]$, so β has a stable, dimensionless interpretation. With $R = D^\top D$, the magnitudes depend on the task-vector norms, and β must be tuned accordingly or R rescaled. Since $s_i > 0$ always and p has at least one positive entry on Δ_m , the vector $\tilde{\lambda}$ has positive sum and the normalization is always well defined.

Definition 4.6 (P2: Task-vector PCGrad with simplex reconstruction). Initialize preference-weighted task directions by

$$d_i := p_i \delta_i, \quad i = 1, \dots, m.$$

Process conflicting pairs in a fixed deterministic order. Whenever a current pair satisfies

$$d_i^\top d_j < 0,$$

update

$$d_i \leftarrow d_i - \frac{d_i^\top d_j}{\|d_j\|_2^2 + \varepsilon} d_j, \quad \varepsilon \geq 0.$$

After all chosen projections, aggregate

$$d_{\text{PC}} := \sum_{i=1}^m d_i.$$

Since d_{PC} need not correspond to a valid convex merge coefficient, reconstruct coefficients by solving

$$\lambda_{\text{P2}} := \arg \min_{\lambda \in \Delta_m} \|D\lambda - d_{\text{PC}}\|_2^2 + \rho \|\lambda - p\|_2^2.$$

Remark 4.7 (Equivalent R -metric implementation). Let e_i be the i -th standard basis vector and

$$c_i := p_i e_i.$$

Then $Dc_i = p_i \delta_i = d_i$, and

$$c_i^\top R c_j = (Dc_i)^\top (Dc_j).$$

Thus projection can also be performed in the R -induced metric:

$$c_i \leftarrow c_i - \frac{c_i^\top R c_j}{c_j^\top R c_j + \varepsilon} c_j.$$

Remark 4.8 (Heuristic nature). This PCGrad-inspired mapping is heuristic and may depend on projection order. A deterministic ordering by strongest negative conflict can be used for reproducibility.

4.4 CAGrad-inspired coefficient mapping

Definition 4.9 (C1: Trust-Region CAGrad-inspired method). Let $p \in \Delta_m$, let $R \succeq 0$, let $c \geq 0$, and let $\varepsilon > 0$. Define

$$\lambda_{\text{C1}} := \arg \max_{\lambda \in \Delta_m} \min_{i=1, \dots, m} (R\lambda)_i$$

subject to

$$(\lambda - p)^\top R(\lambda - p) \leq c^2 \max(p^\top R p, \varepsilon).$$

Equivalently, introduce $t \in \mathbb{R}$ and solve

$$\max_{\lambda \in \Delta_m, t} t$$

subject to

$$(R\lambda)_i \geq t \quad \forall i, \quad (\lambda - p)^\top R(\lambda - p) \leq c^2 \max(p^\top R p, \varepsilon).$$

Remark 4.10 (Interpretation). If $R = D^\top D$, then

$$(R\lambda)_i = \delta_i^\top D\lambda.$$

Thus the objective protects against worst-case negative alignment in the proxy geometry while the trust region keeps λ close to p . This is a one-shot coefficient mapping inspired by conflict-averse aggregation, not the original CAGrad training algorithm.

Definition 4.11 (C2: Soft-min CAGrad-inspired variant). For $\tau > 0$, define

$$\text{softmin}_\tau(z) := -\tau \log \sum_{i=1}^m \exp(-z_i/\tau).$$

Then

$$\lambda_{\text{C2}} := \arg \max_{\lambda \in \Delta_m} \text{softmin}_\tau(R\lambda) - \rho(\lambda - p)^\top R(\lambda - p).$$

This is a smooth approximation of the worst-case alignment objective; as $\tau \rightarrow 0^+$, softmin_τ converges to the minimum, recovering the C1 objective.

Definition 4.12 (C3: Preference-Alignment CAGrad-inspired Mapping). Given $\mathbf{p} \in \Delta_m$, $\mathbf{R} \succeq 0$, and $\rho > 0$, define

$$\lambda_{\text{C3}}^* = \arg \max_{\lambda \in \Delta_m} \mathbf{p}^\top \mathbf{R} \lambda - \rho(\lambda - \mathbf{p})^\top \mathbf{R}(\lambda - \mathbf{p}).$$

C3 is CAGrad-inspired because it searches for a corrected merge direction that remains aligned with the preference-induced reference direction while penalizing deviation from the original preference vector. The first term maximizes geometric alignment between the direct preference direction and the corrected merge direction, whereas the second term measures preference deviation in the $\mathbf{R}$ -metric.

Remark 4.13 (Task-vector interpretation and distinction). If $\mathbf{R} = \Delta^\top \Delta$, where Δ denotes the task-vector matrix, then

$$\mathbf{p}^\top \mathbf{R} \lambda = (\Delta \mathbf{p})^\top (\Delta \lambda).$$

Thus the linear term measures alignment directly in task-vector space. In contrast, C1 protects hard worst-case alignment and C2 uses a soft-min approximation of worst-case alignment; C3 instead maximizes preference-direction alignment. Its main limitation is that it does not explicitly protect the worst-aligned objective, but optimizes preference-weighted alignment instead.

Chapter 5

Practical LoRA-Based Implementation

LoRA is a practical implementation choice here, not the theoretical contribution and not part of original Rewarded Soups.

5.1 LoRA definition

Let $\mathcal{L}$ be the set of adapted linear layers. For layer $\ell \in \mathcal{L}$, let W_ℓ denote the frozen base weight. For objective i , define

$$W'_{i,\ell} = W_\ell + \Delta W_{i,\ell}, \quad \Delta W_{i,\ell} = B_{i,\ell} A_{i,\ell}.$$

Optionally, with the usual scaling,

$$\Delta W_{i,\ell} = \frac{\alpha_\ell}{r_\ell} B_{i,\ell} A_{i,\ell}.$$

Define the effective multi-layer LoRA update

$$\Delta_i^{\text{LoRA}} := (\Delta W_{i,\ell})_{\ell \in \mathcal{L}},$$

and define the flattened LoRA task vector by

$$\delta_i^{\text{LoRA}} := \text{vec}\left(\text{concat}_{\ell \in \mathcal{L}} \Delta W_{i,\ell}\right) \in \mathbb{R}^{d_{\text{LoRA}}}.$$

5.2 LoRA geometry and merging

Define the unnormalized LoRA relationship matrix by

$$R_{ij} = (\delta_i^{\text{LoRA}})^\top \delta_j^{\text{LoRA}}$$

and the cosine LoRA relationship matrix by

$$\hat{R}_{ij} = \cos(\delta_i^{\text{LoRA}}, \delta_j^{\text{LoRA}}) = \frac{(\delta_i^{\text{LoRA}})^\top \delta_j^{\text{LoRA}}}{\|\delta_i^{\text{LoRA}}\|_2 \|\delta_j^{\text{LoRA}}\|_2}.$$

The effective updates need not be materialized as dense matrices. For two adapters (i) and (j), their Frobenius inner product can be accumulated layerwise as

$$\langle \Delta W_{i,\ell}, \Delta W_{j,\ell} \rangle_F = s_{i,\ell} s_{j,\ell} \operatorname{tr}((B_{i,\ell} A_{i,\ell})^\top B_{j,\ell} A_{j,\ell}), \quad s_{i,\ell} := \frac{\alpha_{i,\ell}}{r_{i,\ell}}.$$

Equivalently, this quantity can be evaluated from the small factors as $s_{i,\ell} s_{j,\ell} \langle B_{i,\ell}^\top B_{j,\ell}, A_{i,\ell} A_{j,\ell}^\top \rangle_F$. Consequently, the relationship matrix is invariant under any invertible factor reparameterization $B_{i,\ell} \mapsto B_{i,\ell} G$ and $A_{i,\ell} \mapsto G^{-1} A_{i,\ell}$ that leaves the effective update unchanged. Raw concatenations of the A - and B -factors are therefore not used to construct R .

For $\lambda \in \Delta_m$, define the merged effective LoRA update layerwise by

$$\Delta W_{\text{merged},\ell}^{\text{LoRA}} = \sum_{i=1}^m \lambda_i \Delta W_{i,\ell}.$$

Equivalently,

$$\Delta_{\text{merged}}^{\text{LoRA}} = \sum_{i=1}^m \lambda_i \Delta_i^{\text{LoRA}}.$$

Remark 5.1 (Important caveat). In general,

$$\left(\sum_{i=1}^m \lambda_i B_{i,\ell} \right) \left(\sum_{i=1}^m \lambda_i A_{i,\ell} \right) \neq \sum_{i=1}^m \lambda_i B_{i,\ell} A_{i,\ell}.$$

Therefore the thesis must distinguish between linearly combining *effective LoRA updates* and naively averaging A_i and B_i separately.

Chapter 6

Experimental Setup

6.1 Candidate datasets

Possible datasets for the thesis include:

- Anthropic HH-RLHF for helpfulness and harmlessness,
- UltraFeedback and HelpSteer for richer many-objective alignment settings.

TODO: Specify the exact subset, splits, reward models, preprocessing, and evaluation prompts.

6.2 First prototype

For a first prototype:

- use GPT-2 or TinyLlama as the shared base model,
- train separate LoRA adapters from the same base model,
- start with two objectives from HH-RLHF: helpfulness and harmlessness,
- merge adapters for different $\lambda \in \Delta_2$,
- compute R from adapter geometry,
- later extend to more objectives.

Evaluation should compare $\lambda = p$ against corrected $\lambda = f(p, R)$ and aggregate results over $\mathcal{P} \subset \Delta_m$.

TODO: Add the concrete compute budget, LoRA ranks, search sets $\mathcal{B}$, and held-out evaluation protocol.

Chapter 7

Results

This draft contains no experimental results.

Chapter 8

Discussion

Chapter 9

Conclusion

This thesis draft formalizes the problem around the pipeline

$$(\delta_i)_{i=1}^m \longrightarrow R \longrightarrow \lambda = f(p, R) \longrightarrow \theta(\lambda),$$

with all methods restricted to the same fixed interpolation family.

Bibliography

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. *arXiv preprint arXiv:2110.14048*, 2021.
- Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston, 1999.
- Alexandre Ramé, Guillaume Couairon, Mustafa Shukor, Corentin Dancette, Jean-Baptiste Gaya, Laure Soulier, and Matthieu Cord. Rewarded soups: Towards pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards. *arXiv preprint arXiv:2306.04488*, 2023.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. *arXiv preprint arXiv:1810.04650*, 2018.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2023.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *arXiv preprint arXiv:2001.06782*, 2020.